

Face Tracking with Identity Lock

Smile Blink and Position Detection

Part 2

Gabriel Baziramwabo

Benax Technologies Ltd · Rwanda Coding Academy

Table of Contents

About Part 2.....	3
Chapter 1 From Recognition to Tracking.....	4
Chapter 2 Identity Lock.....	5
Chapter 3 Tracking the Locked Face.....	7
Chapter 4 Smile Blink and Closed Eye Detection.....	9
Chapter 5 Building the Complete Application.....	11
Chapter 6 Testing and Calibration.....	13
Chapter 7 Preparing the Signals for Part 3.....	15
Chapter 8 Final Takeaways.....	16
References.....	17
Appendix A Code.....	18

About Part 2

Part 1 ended with a live ArcFace ONNX recognition pipeline. Part 2 keeps that identity result and adds continuity. The system locks one enrolled person, follows that person across frames, ignores every other face, detects a smile, counts blinks, distinguishes closed eyes from a blink, and reports whether the locked face is above, below, left, right, or near the center of the frame.

The final output is a stable software signal. It does not move a motor. Part 3 will consume the horizontal and vertical error values and convert them into commands for servo or stepper motors so that the camera can bring the locked face back to the middle of the frame.

What You Will Build

- A target-specific lock that accepts only one enrolled identity.
- A lightweight tracker that keeps continuity between identity checks.
- A landmark-based smile detector with hysteresis.
- A blink detector and a separate sustained eye-closed state.
- Normalized horizontal and vertical position errors for the next motor-control stage.
- A practical test procedure for distractors, occlusion, lighting, and threshold calibration.

Starting Point

Continue from the Part 1 project. The following files and data must already work: `src/align.py`, `src/recognize.py`, `models/embedder_arcface.onnx`, and `data/db/face_db.npz`. At least one identity must be enrolled. Run the Part 1 recognition program first and confirm that the intended target is recognized reliably before adding tracking.

Chapter 1 From Recognition to Tracking

Recognition and Tracking Solve Different Problems

Recognition answers who is in the frame. Tracking answers where the same person is now. A recognition system may correctly identify a face in one frame and then repeat the entire decision in the next frame. A tracker connects those observations over time and preserves one target as the camera scene changes.

Stage	Question	Typical output
Detection	Where are all visible faces?	A list of bounding boxes
Recognition	Whose face is this?	Name, similarity, accepted or rejected
Identity lock	Which accepted person may control the system?	One target identity
Tracking	Where is that target now?	Current box and center point
Signal extraction	What is the target doing and where is the target?	Smile, eye state, x error, y error

The Part 2 Pipeline



Only the locked identity reaches tracking and gesture analysis

Figure 1 Recognition is the gate before tracking and facial signal analysis

The identity lock is the boundary that protects every downstream result. A face may be visible and may even be recognized as another enrolled person, but it does not reach the tracking or gesture stages unless its accepted name matches the requested target.

Why Recognition Should Not Run on Every Frame

ArcFace embedding extraction is more expensive than comparing nearby boxes. After the target is locked, the system uses geometric continuity on most frames and periodically rechecks identity. This keeps CPU use manageable while reducing the chance that the tracker transfers to a person who crosses the target's path.

Periodic verification is a safety check, not a new lock decision. If the associated face fails verification, the tracker rejects that candidate. It does not switch to the nearest visible face.

Coordinate Convention

OpenCV image coordinates begin at the top-left corner. The x value increases to the right and the y value increases downward. Therefore, a negative horizontal error means the face is left of center, while a negative vertical error means the face is above center. Part 3 must map these image errors to the actual motor directions of the assembled mechanism.

Chapter 2 Identity Lock

The Lock Contract

A strict lock has one rule: only the requested enrolled identity can become the active target. The program receives the target name on the command line. While searching, it may inspect several faces, but it accepts only a match whose name equals the target and whose ArcFace distance passes the Part 1 threshold.

- One active identity at a time.
- Unknown faces never acquire the lock.
- Other enrolled identities never replace the target.
- A temporary missed detection preserves the target for a short grace period.
- A timeout clears geometry and searches again for the same target name.

Three Lock States

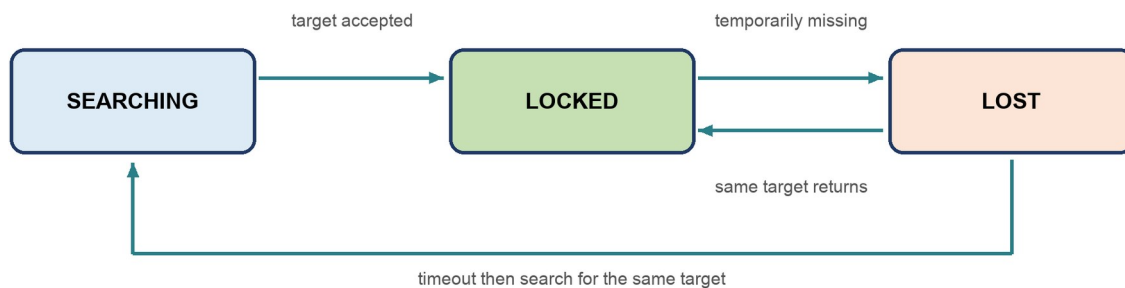


Figure 2 The lock returns to searching only after the target has been missing for the timeout

State	Meaning	Allowed transition
SEARCHING	No target geometry is trusted	LOCKED only when the requested identity is accepted
LOCKED	The target is visible and associated	LOST when the expected face is temporarily missing
LOST	The target identity is retained but no current output is produced	LOCKED if the same target returns; SEARCHING after timeout

Lock Acquisition

1. Detect up to several candidate faces.
2. Align each candidate with the five landmarks from Part 1.
3. Extract an ArcFace embedding and compare it with the enrolled database.
4. Discard Unknown and every accepted name that is not the requested target.
5. If more than one target candidate survives, choose the one with the highest similarity.

Lock Retention

Between identity checks, the tracker associates the new detections with the previous target box. The score combines intersection over union with normalized center displacement. A nearby overlapping box is preferred; a distant box receives a penalty. This calculation is fast and makes no claim about identity on its own.

Every `verify_every` frames, the selected box must again match the locked identity. This matters when two people cross, when the target leaves, or when the detector produces a new box after an occlusion.

Missing Faces and Reacquisition

A single missed frame should not destroy the lock. The LOST state holds the identity for `lost_timeout` frames but produces no movement or facial-expression signal during that gap. If a geometrically plausible face returns, periodic identity verification confirms it. After the timeout, the tracker discards the old box and searches the full frame for the same target name.

Distractor Test

Place the target and a second person in view. Start the program with the target's enrolled name. The displayed box should appear only around the target. Ask the second person to move closer to the camera and cross in front of the target. The expected result is a temporary LOST state or continued target tracking, never a transfer of the lock to the distractor.

Chapter 3 Tracking the Locked Face

The Bounding Box Center

For a box $(x1, y1, x2, y2)$, the face center is $cx = (x1 + x2) / 2$ and $cy = (y1 + y2) / 2$. The raw center changes slightly even when a person holds still because the detector does not return exactly the same box on every frame.

Exponential Smoothing

An exponential moving average reduces this jitter. With $\alpha = 0.30$, the new smoothed center keeps 30 percent of the latest observation and 70 percent of the previous smoothed value. A larger alpha responds faster but passes more jitter. A smaller alpha looks steadier but adds delay.

$$\text{smoothed} = \alpha \times \text{current} + (1 - \alpha) \times \text{previous}$$

Normalized Position Error

Pixel error depends on camera resolution. Normalized error does not. The horizontal error divides the distance from the frame center by half the frame width. The vertical calculation uses half the frame height. Values near -1 and $+1$ represent the edges of the image; zero represents the center.

Signal	Calculation	Interpretation
error_x	$(cx - \text{frame_width} / 2) / (\text{frame_width} / 2)$	Negative is left; positive is right
error_y	$(cy - \text{frame_height} / 2) / (\text{frame_height} / 2)$	Negative is up; positive is down

Center Dead Zone

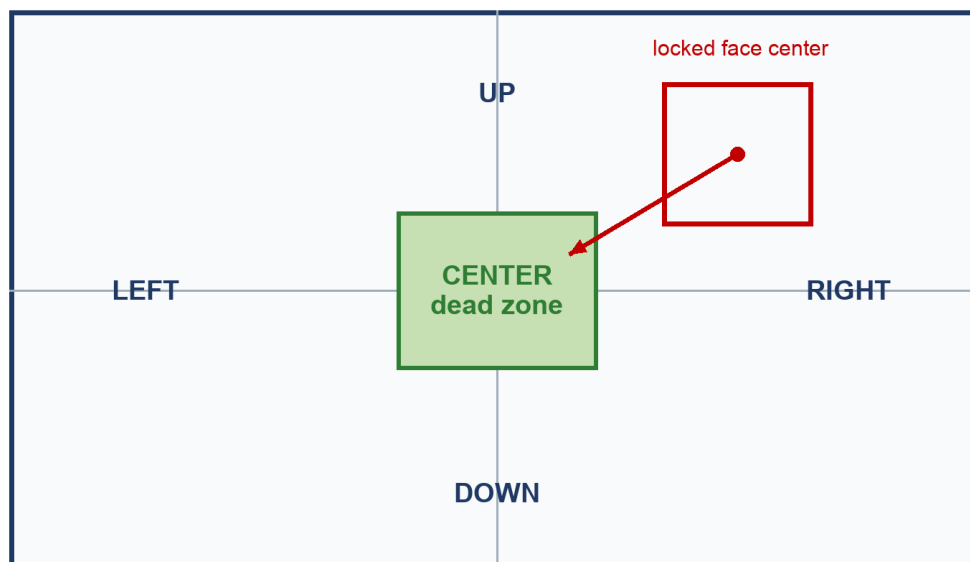


Figure 3 Position labels come from the locked face center and a small center dead zone

A dead zone prevents rapid LEFT and RIGHT changes when the face is already close to the middle. With `dead_zone = 0.07`, an error between -0.07 and $+0.07$ is reported as CENTER on that axis. Part 3 can reuse the same idea before commanding a motor.

Combined Direction

Horizontal and vertical labels remain separate. A face can be RIGHT and UP at the same time. Keeping the axes separate produces cleaner inputs for a pan-and-tilt mechanism and avoids a long list of diagonal labels.

Tracking Is Not Head Pose

This chapter measures the location of the face in the image. It does not estimate whether the person has turned the head left, right, up, or down. Head-pose estimation uses facial geometry and a camera model. Here, LEFT means that the locked face center is on the left side of the frame.

Chapter 4 Smile Blink and Closed Eye Detection

Why Five Points Are Not Enough

The five landmarks used for ArcFace alignment locate the eyes, nose, and mouth corners. They are sufficient for alignment but not for eye opening or lip shape. Part 2 therefore runs MediaPipe FaceMesh on the locked face region and uses the detailed landmarks only for that one person.

Eye Aspect Ratio

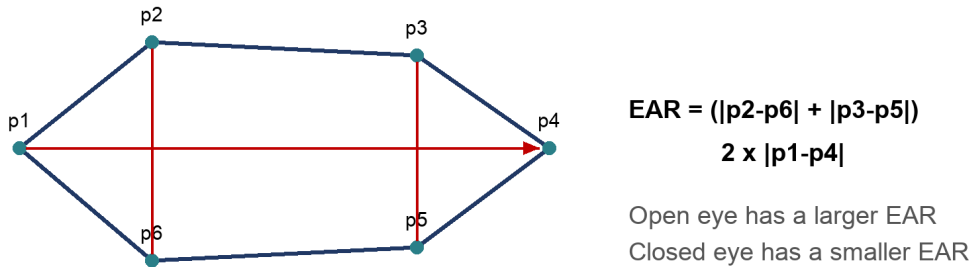


Figure 4 Eye aspect ratio falls when the eyelids move closer together

Eye aspect ratio, abbreviated EAR, compares two vertical eyelid distances with the horizontal eye width. The ratio is fairly stable for an open eye and falls when the eye closes. The program averages the left and right values so that one noisy landmark has less influence.

Blink Event and Closed Eye State

A blink is an event. Closed eyes are a state. The distinction depends on duration. When EAR remains below the threshold for at least `blink_min_frames` and then rises, the code emits one blink event. When EAR remains low for `closed_frames`, the program reports EYES CLOSED. It does not count a new blink on every closed frame.

Observation	Condition	Output
Normal open eyes	EAR is above the threshold	EYES OPEN
Short closure followed by reopening	Low EAR for at least 2 frames, then recovery	One BLINK event
Sustained closure	Low EAR for at least 8 frames	EYES CLOSED
Landmarks unavailable	FaceMesh fails inside the locked box	No eye or smile output for that frame

Smile Score

The smile score divides mouth width by face width. Normalization makes the score less sensitive to the person's distance from the camera. The default threshold is only a starting point because facial proportions, camera angle, and landmark quality vary between people.

The detector uses two thresholds. `smile_on` turns the state on, while the slightly lower `smile_off` turns it off. This hysteresis prevents the label from flickering when the score sits near one threshold.

Calibration Procedure

1. Keep the target near the center under ordinary lighting.
2. Record EAR while the target looks normally at the camera, then during several natural blinks.
3. Choose an EAR threshold between the open-eye values and the lowest blink values.
4. Record the smile score during a neutral expression and several smiles.
5. Set smile_on above the neutral range and smile_off slightly below smile_on.
6. Repeat the test with glasses, moderate head movement, and the intended camera distance.

Limits of Geometric Expression Detection

These signals describe landmark geometry; they do not infer emotion, attention, fatigue, or intent. A wide mouth may resemble a smile, and downward gaze may reduce EAR. Use the output as an interaction signal only after testing it with the intended users and conditions.

Chapter 5 Building the Complete Application

Files Added in Part 2

```
src/
├── face_signals.py    # smile, blink, and eye-closed logic
└── face_tracking.py  # identity lock, tracking, and position output
```

The new files import alignment, detection, embedding, database loading, and matching from Part 1. This avoids copying the recognition pipeline and keeps identity behavior consistent across both parts.

Dependency Check

Use the same virtual environment as Part 1. The implementation expects OpenCV, NumPy, ONNX Runtime, and the same MediaPipe version used by the existing project. If Part 1 was created exactly as written, no new package is required.

```
python -m pip install opencv-python numpy onnxruntime mediapipe==0.10.21
```

Add the Source Files

1. Create `src/face_signals.py` and copy Appendix A1 into it.
2. Create `src/face_tracking.py` and copy Appendix A2 into it.
3. Confirm that `models/embedder_arcface.onnx` and `data/db/face_db.npz` exist.
4. Use the exact enrolled spelling of the target name when starting the program.

Run the Program

```
python -m src.face_tracking --target "Gabriel"
```

Replace Gabriel with an identity stored in the Part 1 database. Press q to quit. The main window displays the lock state, the target box, horizontal and vertical position, normalized error, smile state, eye state, and accumulated blink count.

Expected Behavior

- SEARCHING appears until the requested identity is accepted.
- Only the locked target receives a visible bounding box.
- Other faces may remain in the image but receive no labels or gesture analysis.
- A brief occlusion changes the state to LOST without transferring the lock.
- A long absence returns the system to SEARCHING for the same identity.
- The direction labels change as the locked face crosses the center dead zone.

Key Parameters

Parameter	Default	Effect
threshold	0.34 distance	Identity acceptance threshold from Part 1 evaluation
verify_every	10 frames	Frequency of ArcFace re-verification while locked
lost_timeout	24 frames	Grace period before a full-frame target search

Parameter	Default	Effect
ema_alpha	0.30	Tracking responsiveness versus smoothness
dead_zone	0.07	Centered range on each normalized axis
ear_threshold	0.21	Starting point for eye closure
closed_frames	8 frames	Duration required for EYES CLOSED
smile_on and smile_off	0.38 and 0.35	Smile hysteresis; calibrate for the target

CPU Performance

Face detection still runs on each frame in this teaching implementation. ArcFace verification runs less often after lock, and detailed FaceMesh analysis runs only inside the locked face region. If the frame rate is low, reduce camera resolution first. More advanced optical flow or correlation tracking can be added later without changing the lock contract.

Chapter 6 Testing and Calibration

Test One Variable at a Time

Validate identity, continuity, position, eyes, and smile separately before combining difficult conditions. A test is useful only when its expected result is stated in advance.

Test	Action	Pass condition
Target acquisition	Target enters alone	LOCKED appears with the correct name
Unknown person	An unenrolled person enters	No lock is acquired
Wrong enrolled identity	A different enrolled person enters	No lock is acquired
Distractor crossing	Another face crosses the locked box	No identity transfer
Brief occlusion	Cover the target for under the timeout	LOST, then LOCKED on the same target
Long absence	Target leaves beyond the timeout	SEARCHING resumes for the same target
Position	Move across all four frame regions	Labels and error signs follow image coordinates
Blink	Blink naturally five times	Five events with few or no extra counts
Closed eyes	Hold both eyes closed	EYES CLOSED appears after the configured duration
Smile	Alternate neutral and smiling	Stable state changes without rapid flicker

Threshold Tuning Order

1. Use the evaluated ArcFace threshold from Part 1. Do not compensate for poor recognition by tuning tracker geometry.
2. Tune `verify_every` and `lost_timeout` with crossing and occlusion tests.
3. Tune `ema_alpha` and `dead_zone` while the target moves slowly around the center.
4. Tune EAR and duration thresholds from recorded open, blink, and closed-eye values.
5. Tune smile thresholds last, after the target box and FaceMesh landmarks are stable.

Common Problems

Problem	Likely cause	Adjustment
Lock changes to another face	Verification is too infrequent or association is too permissive	Lower <code>verify_every</code> ; tighten association gate
Target becomes LOST during fast motion	Center displacement penalty is too strong	Permit a larger displacement or increase camera FPS
Box jitters near center	Raw detector movement reaches the label	Lower <code>ema_alpha</code> or widen <code>dead_zone</code> slightly

Problem	Likely cause	Adjustment
Blinks are missed	EAR threshold is too low or minimum duration is too long	Measure EAR and adjust one parameter
Blinks count while eyes remain closed	Event and state logic were combined	Count only on the transition back to open
Smile label flickers	No hysteresis or thresholds are too close	Increase the gap between smile_on and smile_off
Signals appear on another person	FaceMesh is running on the full frame	Run detailed analysis only inside the locked box

Logging for Repeatable Tests

For calibration, record timestamp, lock state, target name, error_x, error_y, EAR, smile score, and event flags to a CSV file. Video and numeric logs make threshold decisions reproducible. Avoid storing face images unless the project has a clear consent, retention, and access policy.

Chapter 7 Preparing the Signals for Part 3

The Software and Motor Boundary

Part 2 ends at a clean interface. The vision loop produces normalized horizontal and vertical error for the locked identity. Part 3 will translate those errors into motor movement. Keeping the boundary explicit allows camera vision to be tested without connected hardware and motor control to be tested with simulated error values.

```
TrackingSignal(
    error_x=-0.24,
    error_y=+0.11,
    horizontal="LEFT",
    vertical="DOWN",
)
```

What Part 3 Must Decide

- Which image-error sign corresponds to each physical motor direction.
- How far a servo should rotate for a given normalized error.
- How many stepper pulses are required and how limits are enforced.
- How acceleration, speed limits, and dead zones prevent oscillation.
- What happens when the face is LOST or the camera reaches a mechanical limit.

Safe Handoff Behavior

When the tracker produces no current target, the future motor controller should stop issuing corrective motion. LOST is not a direction. The controller should also ignore smile and eye events when identity is not locked. These rules prevent stale visual information from driving hardware.

Recommended Message Format

A simple serial or network message can carry the state without sending images. Include a sequence number or timestamp so the receiver can reject stale messages.

```
{"locked": true, "name": "Gabriel",
  "error_x": -0.24, "error_y": 0.11,
  "smile": false, "blink": true, "eyes_closed": false}
```

Part 3 should use error_x and error_y for centering. Smile, blink, and closed-eye fields may trigger separate interactions, but they should not alter identity or select a different face.

Chapter 8 Final Takeaways

- Recognition establishes identity; tracking preserves continuity.
- The lock is target-specific and does not transfer to another visible face.
- Geometric association is fast, while periodic ArcFace checks protect identity.
- Blink events and sustained eye closure require different temporal logic.
- Smile detection becomes steadier with normalization and hysteresis.
- Normalized image errors create a resolution-independent interface for Part 3.
- Motor movement must stop when the locked face has no current observation.

After completing Part 2, the program can identify one allowed person, keep that person as the only target, describe the target's position, and detect simple facial gestures. The next part can focus entirely on controlled mechanical compensation.

References

1. Part 1. Face Recognition with ArcFace ONNX and 5-Point Alignment. Project architecture, enrollment, threshold evaluation, and live recognition.
2. OpenCV documentation. Video capture, drawing functions, image coordinates, and Haar cascade detection.
3. MediaPipe Face Mesh documentation. Dense facial landmark estimation and landmark indexing.
4. Soukupová, T., and Čech, J. Real-Time Eye Blink Detection Using Facial Landmarks. 2016.
5. ArcFace. Additive Angular Margin Loss for Deep Face Recognition. Identity embedding method used by the Part 1 ONNX pipeline.

Privacy and Responsible Use

Face embeddings and camera footage are sensitive biometric data. Enroll people with informed permission, keep the database access-controlled, define how long data is retained, and provide a way to remove an identity. Do not use smile or eye geometry to infer a person's emotional state, attentiveness, or health.

Appendix A Code

The listings below are designed to be copied into the Part 1 project. They depend on the public classes and functions defined in the Part 1 `src/recognize.py` and `src/align.py` files.

A1 `src/face_signals.py`

```
# src/face_signals.py
from dataclasses import dataclass
from typing import Optional, Tuple

import cv2
import mediapipe as mp
import numpy as np

LEFT_EYE = (33, 160, 158, 133, 153, 144)
RIGHT_EYE = (362, 385, 387, 263, 373, 380)
MOUTH_LEFT, MOUTH_RIGHT = 61, 291
LIP_TOP, LIP_BOTTOM = 13, 14
FACE_LEFT, FACE_RIGHT = 234, 454

def distance(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.linalg.norm(a - b))

def eye_aspect_ratio(points: np.ndarray, idx: Tuple[int, ...]) -> float:
    p1, p2, p3, p4, p5, p6 = (points[i] for i in idx)
    width = max(distance(p1, p4), 1e-6)
    return (distance(p2, p6) + distance(p3, p5)) / (2.0 * width)

@dataclass
class FaceSignals:
    ear: float
    blink: bool
    eyes_closed: bool
    smile_score: float
    smiling: bool

class FaceSignalExtractor:
    def __init__(
        self,
        ear_threshold: float = 0.21,
        blink_min_frames: int = 2,
        blink_max_frames: int = 7,
        closed_frames: int = 8,
        smile_on: float = 0.38,
        smile_off: float = 0.35,
    ):
        self.ear_threshold = ear_threshold
        self.blink_min_frames = blink_min_frames
        self.blink_max_frames = blink_max_frames
        self.closed_frames = closed_frames
        self.smile_on = smile_on
        self.smile_off = smile_off
        self.low_ear_frames = 0
        self.smiling = False
        self.mesh = mp.solutions.face_mesh.FaceMesh(
            static_image_mode=False,
            max_num_faces=1,
            refine_landmarks=True,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        )

    def reset(self) -> None:
        self.low_ear_frames = 0
        self.smiling = False
```

```

def close(self) -> None:
    self.mesh.close()

def analyze(self, frame: np.ndarray, bbox) -> Optional[FaceSignals]:
    h, w = frame.shape[:2]
    x1, y1, x2, y2 = bbox
    bw, bh = x2 - x1, y2 - y1
    pad_x, pad_y = int(0.12 * bw), int(0.18 * bh)
    rx1, ry1 = max(0, x1 - pad_x), max(0, y1 - pad_y)
    rx2, ry2 = min(w, x2 + pad_x), min(h, y2 + pad_y)
    roi = frame[ry1:ry2, rx1:rx2]
    if roi.size == 0:
        return None

    result = self.mesh.process(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
    if not result.multi_face_landmarks:
        return None

    rh, rw = roi.shape[:2]
    lm = result.multi_face_landmarks[0].landmark
    points = np.array(
        [[p.x * rw + rx1, p.y * rh + ry1] for p in lm],
        dtype=np.float32,
    )

    left_ear = eye_aspect_ratio(points, LEFT_EYE)
    right_ear = eye_aspect_ratio(points, RIGHT_EYE)
    ear = 0.5 * (left_ear + right_ear)

    blink = False
    if ear < self.ear_threshold:
        self.low_ear_frames += 1
    else:
        if self.blink_min_frames <= self.low_ear_frames <= self.blink_max_frames:
            blink = True
        self.low_ear_frames = 0
    eyes_closed = self.low_ear_frames >= self.closed_frames

    face_width = max(distance(points[FACE_LEFT], points[FACE_RIGHT]), 1e-6)
    mouth_width = distance(points[MOUTH_LEFT], points[MOUTH_RIGHT])
    smile_score = mouth_width / face_width
    if self.smiling:
        self.smiling = smile_score >= self.smile_off
    else:
        self.smiling = smile_score >= self.smile_on

    return FaceSignals(
        ear=ear,
        blink=blink,
        eyes_closed=eyes_closed,
        smile_score=smile_score,
        smiling=self.smiling,
    )

```

A2 src face_tracking.py

```
# src/face_tracking.py
import argparse
from dataclasses import dataclass
from enum import Enum, auto
from pathlib import Path
from typing import Optional

import cv2
import numpy as np

from src.align import align_face_5pt
from src.face_signals import FaceSignalExtractor
from src.recognize import (
    ArcFaceEmbedderONNX,
    FaceDBMatcher,
    HaarFaceMesh5pt,
    load_db_npz,
)

class LockState(Enum):
    SEARCHING = auto()
    LOCKED = auto()
    LOST = auto()

def iou(a, b) -> float:
    ax1, ay1, ax2, ay2 = a
    bx1, by1, bx2, by2 = b
    ix1, iy1 = max(ax1, bx1), max(ay1, by1)
    ix2, iy2 = min(ax2, bx2), min(ay2, by2)
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = max(1, (ax2 - ax1) * (ay2 - ay1))
    area_b = max(1, (bx2 - bx1) * (by2 - by1))
    return inter / float(area_a + area_b - inter)

def center(box):
    x1, y1, x2, y2 = box
    return np.array([(x1 + x2) / 2.0, (y1 + y2) / 2.0], dtype=np.float32)

@dataclass
class TrackingSignal:
    error_x: float
    error_y: float
    horizontal: str
    vertical: str

class LockedFaceTracker:
    def __init__(
        self,
        target_name: str,
        detector,
        embedder,
        matcher,
        verify_every: int = 10,
        lost_timeout: int = 24,
        ema_alpha: float = 0.30,
        dead_zone: float = 0.07,
    ):
        self.target_name = target_name
        self.detector = detector
        self.embedder = embedder
        self.matcher = matcher
        self.verify_every = verify_every
        self.lost_timeout = lost_timeout
        self.ema_alpha = ema_alpha
        self.dead_zone = dead_zone
        self.state = LockState.SEARCHING
        self.last_box = None
        self.smooth_center = None
```

```

self.lost_frames = 0
self.frame_index = 0

@staticmethod
def box(face):
    return (face.x1, face.y1, face.x2, face.y2)

def identity(self, frame, face):
    aligned, _ = align_face_5pt(frame, face.kps, out_size=(112, 112))
    return self.matcher.match(self.embedder.embed(aligned))

def target_is_verified(self, frame, face) -> bool:
    match = self.identity(frame, face)
    return match.accepted and match.name == self.target_name

def acquire(self, frame, faces):
    best = None
    best_similarity = -1.0
    for face in faces:
        match = self.identity(frame, face)
        if (
            match.accepted
            and match.name == self.target_name
            and match.similarity > best_similarity
        ):
            best, best_similarity = face, match.similarity
    return best

def associate(self, faces):
    if self.last_box is None or not faces:
        return None
    last_center = center(self.last_box)
    last_diag = max(np.linalg.norm(
        np.array([self.last_box[2] - self.last_box[0],
                  self.last_box[3] - self.last_box[1]]), dtype=np.float32
    ), 1.0)
    ranked = []
    for face in faces:
        box = self.box(face)
        overlap = iou(self.last_box, box)
        displacement = np.linalg.norm(center(box) - last_center) / last_diag
        score = overlap - 0.35 * displacement
        ranked.append((score, face))
    score, candidate = max(ranked, key=lambda item: item[0])
    return candidate if score > -0.30 else None

def update(self, frame):
    self.frame_index += 1
    faces = self.detector.detect(frame, max_faces=8)

    if self.state == LockState.SEARCHING:
        candidate = self.acquire(frame, faces)
    else:
        candidate = self.associate(faces)
        if (
            candidate is not None
            and (self.state == LockState.LOST
                 or self.frame_index % self.verify_every == 0)
            and not self.target_is_verified(frame, candidate)
        ):
            candidate = None

    if candidate is None:
        self.lost_frames += 1
        if self.last_box is not None:
            self.state = LockState.LOST
        if self.lost_frames > self.lost_timeout:
            self.state = LockState.SEARCHING
            self.last_box = None
            self.smooth_center = None
        return None, None

    self.state = LockState.LOCKED
    self.lost_frames = 0
    self.last_box = self.box(candidate)
    raw_center = center(self.last_box)

```

```

    if self.smooth_center is None:
        self.smooth_center = raw_center
    else:
        a = self.ema_alpha
        self.smooth_center = a * raw_center + (1.0 - a) * self.smooth_center
    return candidate, self.position_signal(frame.shape)

def position_signal(self, shape) -> TrackingSignal:
    height, width = shape[:2]
    ex = float((self.smooth_center[0] - width / 2.0) / (width / 2.0))
    ey = float((self.smooth_center[1] - height / 2.0) / (height / 2.0))
    horizontal = "CENTER"
    vertical = "CENTER"
    if ex < -self.dead_zone:
        horizontal = "LEFT"
    elif ex > self.dead_zone:
        horizontal = "RIGHT"
    if ey < -self.dead_zone:
        vertical = "UP"
    elif ey > self.dead_zone:
        vertical = "DOWN"
    return TrackingSignal(ex, ey, horizontal, vertical)

def draw_label(frame, text, xy, color, scale=0.62):
    cv2.putText(frame, text, xy, cv2.FONT_HERSHEY_SIMPLEX,
                scale, (0, 0, 0), 4, cv2.LINE_AA)
    cv2.putText(frame, text, xy, cv2.FONT_HERSHEY_SIMPLEX,
                scale, color, 2, cv2.LINE_AA)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", required=True, help="enrolled identity to lock")
    parser.add_argument("--camera", type=int, default=0)
    parser.add_argument("--threshold", type=float, default=0.34)
    args = parser.parse_args()

    detector = HaarFaceMesh5pt(min_size=(70, 70), debug=False)
    embedder = ArcFaceEmbedderONNX(
        model_path="models/embedder_arcface.onnx",
        input_size=(112, 112),
        debug=False,
    )
    matcher = FaceDBMatcher(
        load_db_npz(Path("data/db/face_db.npz")),
        dist_thresh=args.threshold,
    )
    tracker = LockedFaceTracker(args.target, detector, embedder, matcher)
    signals = FaceSignalExtractor()

    cap = cv2.VideoCapture(args.camera)
    if not cap.isOpened():
        raise RuntimeError("Camera not available")

    blink_total = 0
    try:
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            locked_face, position = tracker.update(frame)
            view = frame.copy()

            state_text = f"{tracker.state.name}: {args.target}"
            state_color = (0, 180, 0) if locked_face is not None else (0, 140, 255)
            draw_label(view, state_text, (12, 28), state_color, 0.72)

            if locked_face is not None:
                box = tracker.box(locked_face)
                x1, y1, x2, y2 = box
                cv2.rectangle(view, (x1, y1), (x2, y2), (255, 170, 0), 3)
                face_state = signals.analyze(frame, box)
                if face_state is not None:
                    if face_state.blink:
                        blink_total += 1

```

```

        expression = "SMILE" if face_state.smiling else "NEUTRAL"
        eye_text = "EYES CLOSED" if face_state.eyes_closed else "EYES OPEN"
        draw_label(view, expression, (x1, max(55, y1 - 50)), (0, 255, 255))
        draw_label(view, f"{eye_text} blinks={blink_total}",
                    (x1, max(78, y1 - 25)), (255, 255, 0))
        draw_label(view, f"EAR={face_state.ear:.3f} "
                        f"smile={face_state.smile_score:.3f}",
                    (12, view.shape[0] - 18), (255, 255, 255), 0.52)
        draw_label(view,
                    f"H={position.horizontal} V={position.vertical} "
                    f"error=({position.error_x:+.2f},{position.error_y:+.2f})",
                    (12, 56), (255, 170, 0), 0.60)
    else:
        signals.reset()

    h, w = view.shape[:2]
    dz = tracker.dead_zone
    cv2.rectangle(view,
                  (int(w * (0.5 - dz / 2)), int(h * (0.5 - dz / 2))),
                  (int(w * (0.5 + dz / 2)), int(h * (0.5 + dz / 2))),
                  (120, 120, 120), 1)
    cv2.imshow("Locked Face Tracking", view)
    if (cv2.waitKey(1) & 0xFF) == ord("q"):
        break

finally:
    cap.release()
    signals.close()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

A3 Quick Test Checklist

- ☐ Part 1 live recognition accepts the target reliably.
- ☐ The target name passed to --target matches the database key exactly.
- ☐ Unknown and other enrolled faces cannot acquire the lock.
- ☐ The lock survives a brief missed detection and never transfers during a crossing.
- ☐ LEFT, RIGHT, UP, and DOWN follow the location of the target in the image.
- ☐ Blink count increments once after an eye closure and reopening.
- ☐ EYES CLOSED appears only after a sustained closure.
- ☐ Smile thresholds were calibrated for the target and intended lighting.
- ☐ No movement signal is emitted while the state is LOST or SEARCHING.